

HARDWARE INTEGRATION GUIDE

Smart Bus Garage Management System
MTI University | 3x ESP32 Devices + 6 Sensors & Actuators
Generated April 2026

1. Hardware Overview

1.1 System Architecture Summary

The garage hardware layer consists of three Espressif ESP32 microcontrollers communicating over a shared WiFi LAN (192.168.1.x subnet). All three devices connect to the FastAPI backend running on the same network. The system is self-contained within the local network — no internet connection is required for operation.

1.2 Complete Hardware Inventory

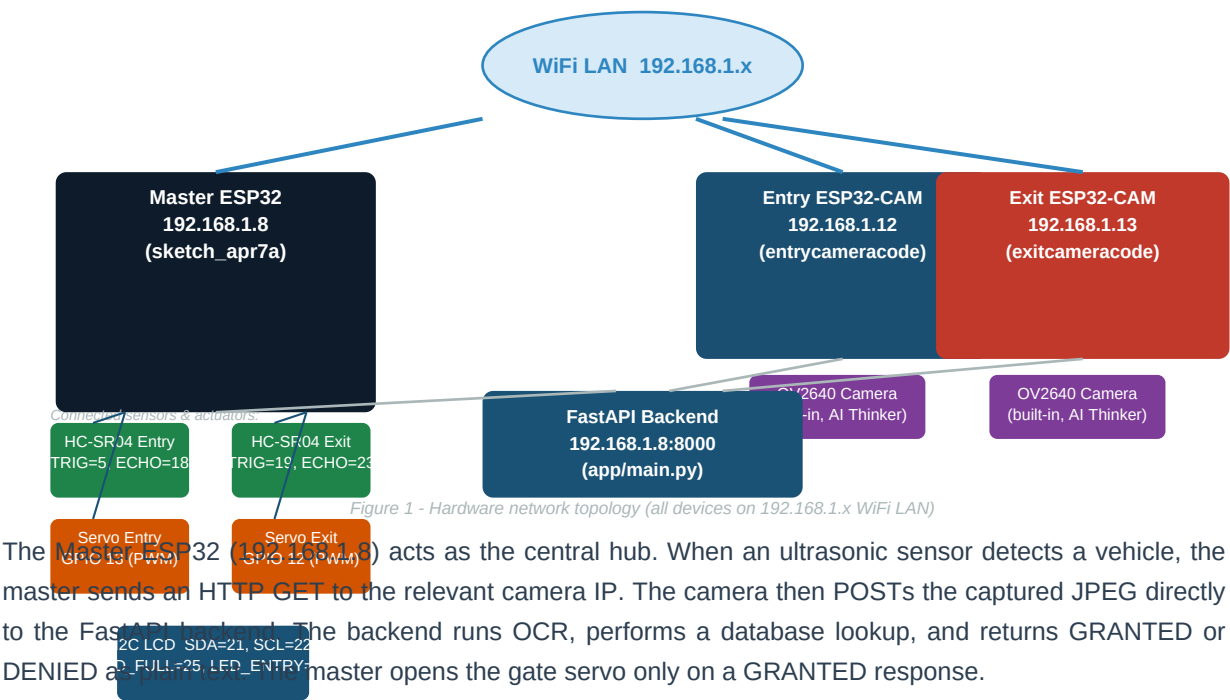
The system comprises three ESP32 microcontrollers plus the following physical sensors and actuators wired to the Master ESP32 and the two camera modules. Every component listed below is required for full end-to-end operation:

Component	Qty	Connected To	Function
Master ESP32 (Generic Dev Module)	1	—	Central controller: reads sensors, drives actuators, triggers cameras via HTTP, persists state
Entry ESP32-CAM (AI Thinker)	1	WiFi LAN	Captures JPEG at entry gate; uploads to backend ANPR pipeline
Exit ESP32-CAM (AI Thinker)	1	WiFi LAN	Captures JPEG at exit gate; uploads to backend ANPR pipeline
HC-SR04 Ultrasonic Sensor — ENTRY	1	Master ESP32 GPIO 5/18	Detects vehicle approaching entry gate (< 10 cm threshold)
HC-SR04 Ultrasonic Sensor — EXIT	1	Master ESP32 GPIO 19/23	Detects vehicle approaching exit gate (< 10 cm threshold)
Servo Motor — Entry Gate	1	Master ESP32 GPIO 13	Physical gate arm: 0 deg = closed, 90 deg = open (4000 ms)
Servo Motor — Exit Gate	1	Master ESP32 GPIO 12	Physical gate arm: 0 deg = closed, 90 deg = open (4000 ms)
I2C LCD Display (16x2)	1	Master SDA=GPIO21, SCL=GPIO22	Shows real-time occupancy: e.g. 'Free Slots: 3'
LED — GARAGE FULL (red)	1	Master ESP32 GPIO 25	Lights when carCount = MAX_CARS (4); entry physically blocked

LED — ENTRY ALLOWED (green)	1	Master ESP32 GPIO 26	Lights when carCount < MAX_CARS; entry open
OV2640 Camera (AI Thinker built-in)	2	Entry/Exit CAM modules	VGA JPEG capture (640x480, quality 12); streams to backend via HTTP POST
FTDI USB-Serial Programmer	1 (shared)	GPIO0 to GND for flash mode	Required to flash both ESP32-CAM boards (not needed for Master ESP32)

Table 1 — Complete hardware inventory (3 ESP32 devices + 9 sensors/actuators)

1.3 Network Topology



The Master ESP32 (192.168.1.8) acts as the central hub. When an ultrasonic sensor detects a vehicle, the master sends an HTTP GET to the relevant camera IP. The camera then POSTs the captured JPEG directly to the FastAPI Backend. The backend runs OCR, performs a database lookup, and returns GRANTED or DENIED as a response. The master opens the gate servo only on a GRANTED response.

2. Data Flow — Sensor to Gate Open

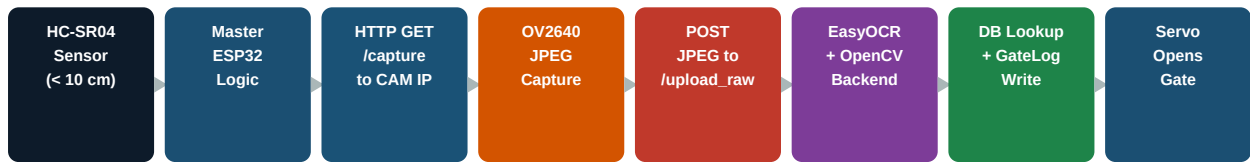


Figure 2 - Complete data flow from physical sensor trigger to gate opening

Step 1 — Vehicle Detection

The HC-SR04 ultrasonic sensor on the Master ESP32 measures the distance continuously. When distance < DETECT_CM (10 cm), the master sets the inDetected flag and calls handleEntryGate() or handleExitGate().

Step 2 — Camera Trigger

Master sends HTTP GET to `http://ENTRY_CAM_IP/capture` (192.168.1.12 for entry, 192.168.1.13 for exit). Timeout is 15,000 ms. The camera's WebServer handles the request on its /capture route.

Step 3 — JPEG Capture

The ESP32-CAM captures a VGA (640x480) JPEG frame using the OV2640 sensor. Up to 3 retry attempts with 300 ms between them. If all fail, `reinitCamera()` is called before one final attempt.

Step 4 — JPEG Upload to Backend

Camera POSTs raw JPEG bytes to `POST /api/v1/hardware/anpr/upload_raw?gate_id=1` (or `gate_id=2` for exit). The `X-Hardware-API-Key` header is included for authentication. `Content-Type` is `image/jpeg`.

Step 5 — OCR Processing

Backend decodes JPEG with OpenCV (`cv2.imdecode`). EasyOCR runs text detection. The highest-confidence detection is selected. A regex strips non-alphanumeric characters (`re.sub('[^A-Z0-9]', '', text.upper())`).

Step 6 — Confidence Check

If OCR confidence ≥ 0.85 (85%): proceed to DB lookup. If confidence < 0.85: event logged as IGNORED, DENIED returned to camera.

Step 7 — Database Lookup

Backend queries the Vehicle table for a matching `plate_number` with status in (EN_ROUTE, ASSIGNED, FREE). If found: event = GRANTED. If not found: event = DENIED.

Step 8 — GateLog Write + WebSocket

A GateLog row is inserted with `gate_id`, `plate_number`, `confidence`, `event`, and `vehicle_id`. `broadcast_to_role('MANAGER')` and `broadcast_to_role('ADMIN')` send a `gate_auth` WebSocket event to all connected dashboards.

Step 9 — Response Chain

Backend returns `PlainTextResponse('GRANTED')` or `PlainTextResponse('DENIED')`. Camera forwards this as HTTP 200 'GRANTED' or HTTP 403 to the master. Master checks for 'GRANTED' substring.

Step 10 — Gate Opens

On GRANTED: `setGateIn(true)` opens the servo. `gateInOpen = true`, `gateInOpenTime = millis()`. `carCount++` is persisted to NVS (Preferences library). LCD updates to show new occupancy count.

Step 11 — Gate Closes (Non-Blocking)

At the top of `loop()`, if `millis() - gateInOpenTime >= GATE_OPEN_MS` (4000 ms), `setGateIn(false)` closes the servo. The gate timer is `millis()`-based so sensor reads and HTTP calls continue during the 4-second open window.

3. GPIO Pin Mapping

The following table documents every physical wire connection in the system. Use this as the reference when wiring or debugging hardware.

Device	GPIO Pin	Connected To	Direction	Notes
Master ESP32	GPIO 5 (TRIG)	HC-SR04 Entry — Trigger	OUT	Entry gate sensor
Master ESP32	GPIO 18 (ECHO)	HC-SR04 Entry — Echo	IN	Entry gate sensor
Master ESP32	GPIO 19 (TRIG)	HC-SR04 Exit — Trigger	OUT	Exit gate sensor
Master ESP32	GPIO 23 (ECHO)	HC-SR04 Exit — Echo	IN	Exit gate sensor
Master ESP32	GPIO 13 (PWM)	Servo — Entry Gate	OUT	LEDC channel, 50 Hz
Master ESP32	GPIO 12 (PWM)	Servo — Exit Gate	OUT	LEDC channel, 50 Hz
Master ESP32	GPIO 21 (SDA)	I2C LCD (16x2)	I2C	LiquidCrystal_I2C
Master ESP32	GPIO 22 (SCL)	I2C LCD (16x2)	I2C	LiquidCrystal_I2C
Master ESP32	GPIO 25	LED — GARAGE FULL	OUT	HIGH when carCount = MAX_CARS
Master ESP32	GPIO 26	LED — ENTRY ALLOWED	OUT	HIGH when space available
Entry CAM	Built-in	OV2640 CMOS Sensor	IN	AI Thinker module
Entry CAM	GPIO 4	Flash LED	OUT	AI Thinker onboard
Exit CAM	Built-in	OV2640 CMOS Sensor	IN	AI Thinker module
Exit CAM	GPIO 4	Flash LED	OUT	AI Thinker onboard

Table 2 — Complete GPIO pin mapping for all three ESP32 devices

4. Key Files and What They Do

The hardware integration spans firmware files (inside the hardware/ directory) and backend Python files (inside app/). The table below describes every file you need to understand or modify to work with the hardware layer.

File / Path	Role	Key Contents
hardware/hardware_config.h	Shared config (all 3 devices)	WIFI_SSID, WIFI_PASSWORD, HW_API_KEY, BACKEND_BASE, ENTRY_CAM_IP, EXIT_CAM_IP, MAX_CARS=4, DETECT_CM=10, CLEAR_CM=15, GATE_OPEN_MS=4000
hardware/sketch_apr7a/.../sketch_apr7a.ino	Master ESP32 firmware	Reads 2x HC-SR04 sensors, drives 2x servos via LEDC PWM, controls LCD + LEDs, triggers cameras over HTTP, persists carCount to NVS
hardware/Entry camera code/.../entrycameracode.ino	Entry ESP32-CAM firmware	Serves HTTP /capture endpoint; captures OV2640 JPEG; POSTs to /anpr/upload_raw with X-Hardware-API-Key; parses GRANTED/DENIED response
hardware/exitcameracode/.../exitcamera_code_copy_20260410082200.ino	Exit ESP32-CAM firmware	Same as entry camera; gate_id=2; fixed off-by-one in GRANTED check (>= 0)
app/api/v1/hardware.py	FastAPI hardware router	POST /log (204), POST /anpr/upload_raw (EasyOCR pipeline, GateLog write, WS broadcast), POST /anpr, POST /iot, POST /gps, POST /camera
app/models/models.py	DB models	GateLog, IotSensorReading, GpsReading, CameraReading, MaintenanceRequest, Vehicle, VehicleStatus
app/core/sockets.py	WebSocket manager	broadcast_to_role() sends gate_auth and iot_alert events to Manager/Admin dashboards in real time

Table 3 — Key files for hardware integration

4.1 hardware_config.h — The Master Configuration File

hardware_config.h is the single source of truth for all credentials and constants. It is #include'd by all three .ino sketch files. A copy lives in each sketch folder so the PlatformIO/Arduino compiler can find it with a simple #include. The file is excluded from git to protect credentials.

```
#define WIFI_SSID "WE_4F038C" — Your WiFi network name
#define WIFI_PASSWORD "d0288c90" — Your WiFi password
#define HW_API_KEY "your_key_from_.env" — Must match HARDWARE_API_KEY in backend .env
#define BACKEND_BASE "http://192.168.1.8:8000/api/v1/hardware" — All backend URLs built from this
#define ENTRY_CAM_IP "192.168.1.12" — Entry ESP32-CAM IP (set fixed in router)
#define EXIT_CAM_IP "192.168.1.13" — Exit ESP32-CAM IP (set fixed in router)
#define MAX_CARS 4 — Physical garage capacity — gate refuses entry when carCount = 4
#define DETECT_CM 10 — HC-SR04 trigger distance in centimetres
#define CLEAR_CM 15 — Hysteresis band — detection flag clears above this distance
#define GATE_OPEN_MS 4000UL — Milliseconds the servo gate stays open
```

5. Remaining Steps to Complete Integration

The firmware and backend code changes are complete. The steps below are the physical and configuration actions needed to make the system fully operational on real hardware.

Step	Action	Status
1	Set real HW_API_KEY in hardware/hardware_config.h (and the 3 sketch-folder copies) — matches HARDWARE_API_KEY in .env	PENDING
2	Flash Master ESP32 via PlatformIO (sketch_apr7a folder) — USB cable, select COM port, click Upload	PENDING
3	Flash Entry ESP32-CAM — GPIO0 to GND while powering on to enter flash mode; remove jumper after upload; press Reset	PENDING
4	Flash Exit ESP32-CAM — same GPIO0 flash procedure as entry camera	PENDING
5	Start backend: uvicorn app.main:app --reload; confirm /hardware/log returns 204	PENDING
6	Open Serial Monitor (115200 baud) for each device; confirm WiFi connects and sendLog() returns 204 (not 403/404)	PENDING
7	Trigger entry camera manually: GET http://192.168.1.12/capture — verify JPEG upload and GRANTED/DENIED response in Serial Monitor	PENDING
8	Full integration test: vehicle approaches sensor -> gate opens -> GateLog row in DB -> Manager dashboard WS event fires	PENDING
9	(Optional) Wire GPS module to UART2 (RX=GPIO16, TX=GPIO17); POST NMEA data to POST /hardware/gps	OPTIONAL
10	(Optional) Wire I2C sensors (ADS1115 ADC) to SDA=GPIO21, SCL=GPIO22; POST readings to POST /hardware/iot	OPTIONAL

Table 4 — Remaining integration steps

Quick Reference: Flash Mode for ESP32-CAM

1. Connect GPIO0 to GND on the ESP32-CAM board using a jumper wire.
2. Power the board (or press Reset) — it is now in flash mode.
3. In VS Code PlatformIO, select the correct COM port and click Upload.
4. Wait for 'Leaving... Hard resetting...' message in the upload terminal.
5. Remove the GPIO0-to-GND jumper.
6. Press the Reset button on the ESP32-CAM board.
7. Open Serial Monitor at 115200 baud — you should see WiFi connecting.